

Universidade Federal de Juiz de Fora
DCC059 – Teoria dos Grafos

Problema da Árvore de Steiner com Coleta de Prêmios

Gabriel Rossini Castro – Matrícula: 202165043A
Luís Otávio Maia Barrientos – Matrícula: 202276009
Yan Vieira Guimarães – Matrícula: 202235015

Julho de 2026

Sumário

1	Introdução	1
2	Descrição do Problema	1
3	Algoritmos Implementados	2
3.1	Algoritmo Guloso	2
3.2	Algoritmo Guloso Randomizado	2
3.3	Algoritmo Guloso Randomizado Reativo	3
4	Experimentos Computacionais	4
4.1	Descrição das instâncias	4
4.2	Ambiente computacional e parâmetros	4
4.3	Resultados obtidos	4
5	Conclusões	6

1 Introdução

Muitos problemas de otimização definidos em grafos não dispõem de algoritmos exatos de tempo polinomial. Nesses casos, heurísticas são uma alternativa prática para produzir soluções de boa qualidade em tempo reduzido. Este trabalho aborda o *Prize-Collecting Steiner Tree Problem* (PCSTP), ou Problema da Árvore de Steiner com Coleta de Prêmios, uma generalização do problema clássico da árvore de Steiner na qual cada vértice possui um prêmio e a solução equilibra o custo de conexão com a penalidade por prêmios não coletados [1].

O objetivo do projeto foi desenvolver e comparar três heurísticas construtivas: uma estratégia gulosa determinística, uma versão gulosa randomizada baseada em Lista Restrita de Candidatos (LRC) e uma versão randomizada reativa. A implementação foi realizada em C++ sobre um tipo abstrato de dados para grafos não direcionados, ponderados nas arestas e nos vértices. Os algoritmos foram avaliados nas instâncias b01, b03 e b08 da biblioteca SteinLib [2], com dez execuções independentes por configuração.

O restante do relatório está organizado da seguinte forma. A Seção 2 apresenta a definição formal do problema. A Seção 3 detalha as três heurísticas implementadas. A Seção 4 descreve as instâncias, o ambiente, os parâmetros e os resultados experimentais. Por fim, a Seção 5 sintetiza as conclusões e indica possibilidades de aprimoramento.

2 Descrição do Problema

Seja $G = (V, E)$ um grafo não direcionado e conexo. Cada aresta $e \in E$ possui custo não negativo c_e , e cada vértice $v \in V$ possui prêmio não negativo p_v . Uma solução é uma árvore $T = (V_T, E_T)$, com $V_T \subseteq V$ e $E_T \subseteq E$. Seu valor objetivo é

$$f(T) = \sum_{e \in E_T} c_e + \sum_{v \in V \setminus V_T} p_v. \quad (1)$$

O primeiro termo de (1) representa o custo das arestas usadas para conectar os vértices selecionados; o segundo representa a penalidade correspondente aos prêmios que deixam de ser coletados. O PCSTP consiste em encontrar uma árvore T que minimize $f(T)$. Assim, um vértice deve ser incluído quando o benefício de coletar seu prêmio compensa o custo necessário para conectá-lo à solução.

As instâncias usadas nos experimentos pertencem originalmente ao problema clássico da árvore de Steiner. Para tratá-las pela mesma implementação, cada terminal recebeu prêmio $M = 10^6$, enquanto os demais vértices receberam prêmio zero. Como M é muito superior aos custos das arestas dessas instâncias, excluir um terminal é sempre desvantajoso. Desse modo, a função (1) passa, na prática, a minimizar o custo da árvore que conecta todos os terminais, tornando os valores comparáveis aos melhores resultados conhecidos da SteinLib.

Na implementação, a solução parcial é ampliada por caminhos mínimos. Para um caminho P que liga a árvore atual a um vértice ainda não selecionado, define-se o ganho incremental por

$$g(P) = \sum_{v \in V(P) \setminus V_T} p_v - \sum_{e \in E(P) \setminus E_T} c_e. \quad (2)$$

Um caminho com $g(P) > 0$ reduz o valor objetivo, pois o prêmio adicional supera o custo de conexão. Os caminhos mínimos entre a árvore parcial e os demais vértices são calculados por uma versão multiorigem do algoritmo de Dijkstra.

3 Algoritmos Implementados

As três abordagens compartilham a mesma construção incremental. A diferença está na escolha da raiz, na seleção do caminho que será inserido e no ajuste do parâmetro de aleatoriedade α .

3.1 Algoritmo Guloso

O algoritmo guloso escolhe como raiz o vértice de maior prêmio. Em seguida, calcula simultaneamente as menores distâncias da árvore parcial para todos os vértices externos. Para cada destino alcançável, reconstrói o caminho mínimo e calcula o ganho definido em (2). O caminho de maior ganho positivo é incorporado à árvore, sem duplicar vértices ou arestas. O processo termina quando nenhum candidato melhora a solução.

Algorithm 1: Construção gulosa

Entrada: Grafo $G = (V, E)$, custos c e prêmios p

Saída: Árvore T

```

1  $r \leftarrow \arg \max_{v \in V} p_v$ ;  $T \leftarrow (\{r\}, \emptyset)$ ;
2 repeat
3   Calcule as menores distâncias de  $V_T$  aos vértices de  $V \setminus V_T$ ;
4   Para cada destino, reconstrua seu caminho mínimo  $P$  até  $T$  e calcule  $g(P)$ ;
5    $P^* \leftarrow \arg \max_P g(P)$ ;
6   if  $g(P^*) > 0$  then
7      $\perp$  adicione a  $T$  os novos vértices e arestas de  $P^*$ ;
8 until nenhum caminho com ganho positivo seja encontrado;
9 retorne  $T$ ;
```

A escolha determinística pode produzir soluções rapidamente, mas é sensível à raiz e aos primeiros caminhos inseridos. Uma decisão local pode tornar caras as conexões futuras e não é reconsiderada, pois não foi implementada uma fase de busca local ou remoção de ramos.

3.2 Algoritmo Guloso Randomizado

A versão randomizada procura reduzir a dependência das primeiras escolhas. Para um conjunto de candidatos com ganhos entre $g_{\min}$ e $g_{\max}$, a LRC é definida por

$$\text{LRC}(\alpha) = \{P : g(P) \geq g_{\max} - \alpha(g_{\max} - g_{\min})\}. \quad (3)$$

Quando $\alpha = 0$, somente os melhores candidatos entram na lista; à medida que α cresce, a lista se torna mais abrangente e a construção mais diversificada. A raiz também é sorteada em uma LRC, formada de maneira análoga com base nos prêmios. Cada iteração constrói uma solução completa e o algoritmo devolve a melhor delas.

Algorithm 2: Guloso randomizado

Entrada: Grafo G , parâmetro α , número de iterações N e semente s

Saída: Melhor árvore encontrada T^*

```
1 Inicialize o gerador Mersenne Twister com  $s$  e defina  $T^* \leftarrow \emptyset$ ;  
2 Construa a LRC de raízes usando os prêmios e  $\alpha$ ;  
3 for  $i \leftarrow 1$  to  $N$  do  
4   sorteie uma raiz na LRC de raízes e inicie  $T$ ;  
5   while existir candidato com ganho positivo do  
6     calcule todos os caminhos candidatos e construa  $\text{LRC}(\alpha)$  por (3);  
7     sorteie  $P \in \text{LRC}(\alpha)$  e acrescente-o a  $T$ ;  
8     if  $f(T) < f(T^*)$  then  
9        $T^* \leftarrow T$ ;  
10 retorne  $T^*$ ;
```

3.3 Algoritmo Guloso Randomizado Reativo

O algoritmo reativo evita fixar previamente um único valor de α . Seja A o conjunto de valores permitidos. Inicialmente, cada $\alpha_i \in A$ recebe probabilidade $1/|A|$. Em cada iteração, um valor é sorteado segundo essas probabilidades e usado na construção randomizada. Ao final de cada bloco, calcula-se a média $\bar{f}_i$ das soluções geradas com α_i e sua qualidade $q_i = 1/\bar{f}_i$. As probabilidades são então atualizadas por

$$\Pr(\alpha_i) = \frac{q_i}{\sum_{j=1}^{|A|} q_j}. \quad (4)$$

Como o PCSTP é de minimização, valores de α que produzem menores médias passam a ter maior probabilidade. Diferentemente da versão randomizada, a implementação reativa mantém como raiz o vértice de maior prêmio e adapta apenas a seleção dos caminhos.

Algorithm 3: Guloso randomizado reativo

Entrada: Grafo G , conjunto A , iterações N , bloco B e semente s

Saída: Melhor árvore encontrada T^*

```
1 Defina  $\Pr(\alpha_i) \leftarrow 1/|A|$  para todo  $\alpha_i \in A$ ;  
2 Escolha como raiz o vértice de maior prêmio;  
3 for  $i \leftarrow 1$  to  $N$  do  
4   sorteie  $\alpha_i$  segundo as probabilidades atuais;  
5   construa  $T$  pelo procedimento randomizado com a raiz fixa e  $\alpha_i$ ;  
6   registre  $f(T)$  e atualize  $T^*$ , se necessário;  
7   if  $i$  for múltiplo de  $B$  then  
8     atualize as probabilidades por (4);  
9 retorne  $T^*$ ;
```

4 Experimentos Computacionais

4.1 Descrição das instâncias

Foram selecionadas três instâncias do conjunto B da SteinLib [2]. Todas são grafos esparsos, não direcionados e com custos inteiros aleatórios nas arestas. A seleção inclui duas instâncias com o mesmo tamanho, mas quantidades distintas de terminais, e uma instância maior. Essa combinação permite observar o efeito do número de terminais e do crescimento do grafo. A Tabela 1 resume suas características; a coluna “melhor conhecido” apresenta o custo de referência registrado para cada instância.

Tabela 1: Características das instâncias utilizadas.

Instância	Vértices	Arestas	Terminais	Melhor conhecido
b01	50	63	9	82
b03	50	63	25	138
b08	75	94	19	104

4.2 Ambiente computacional e parâmetros

Os algoritmos foram implementados em C++ e compilados com o g++ 16.1.0 (MSYS2), em ambiente Windows de 64 bits. Os experimentos foram executados em uma máquina com processador AMD64 e 12 processadores lógicos. A medição de tempo usou `std::chrono::high_resolution_clock`. A aleatoriedade foi produzida por `std::mt19937`; as sementes foram registradas nos arquivos de resultados para permitir a reprodução das execuções.

Cada configuração foi executada dez vezes por instância, com sementes distintas. O guloso randomizado foi testado com $\alpha \in \{0,2, 0,3, 0,4\}$ e 30 construções em cada execução. O reativo realizou 300 construções, usou o mesmo conjunto de valores de α e atualizou as probabilidades em blocos de 30 iterações. Esses valores atendem aos limites mínimos definidos para o experimento.

4.3 Resultados obtidos

Para uma solução de valor z e um melhor valor conhecido z^* , o desvio percentual foi calculado por

$$\text{desvio}(z) = 100 \frac{z - z^*}{z^*}. \quad (5)$$

Logo, desvio zero indica que o algoritmo alcançou o valor de referência, e valores menores são preferíveis. As médias ao final das tabelas atribuem o mesmo peso a cada instância. Em cada linha, os menores desvios entre as abordagens implementadas estão em **negrito**.

Conforme a Tabela 2, todos os métodos atingiram o melhor valor conhecido, 82, na instância b01. Na b03, todos chegaram a 139, apenas uma unidade acima da referência 138, resultando em 0,72% de desvio. A diferenciação ocorreu na instância maior, b08: o

Tabela 2: Desvio percentual do melhor resultado nas dez execuções.

Instância	Guloso	Randomizado			Reativo
		$\alpha = 0,2$	$\alpha = 0,3$	$\alpha = 0,4$	
b01	0,00	0,00	0,00	0,00	0,00
b03	0,72	0,72	0,72	0,72	0,72
b08	3,85	0,96	0,96	2,88	2,88
Média	1,52	0,56	0,56	1,20	1,20

guloso obteve 108, ao passo que os randomizados com $\alpha = 0,2$ e $\alpha = 0,3$ obtiveram 105. A randomização reduziu, portanto, o melhor desvio de 3,85% para 0,96%. Considerando a média dos desvios das três instâncias, essas duas configurações apresentaram o melhor resultado, 0,56%.

Tabela 3: Desvio percentual da média dos resultados das dez execuções.

Instância	Guloso	Randomizado			Reativo
		$\alpha = 0,2$	$\alpha = 0,3$	$\alpha = 0,4$	
b01	0,00	0,00	0,00	0,00	0,00
b03	0,72	0,72	0,72	0,72	0,72
b08	3,85	1,15	3,75	5,10	3,27
Média	1,52	0,63	1,49	1,94	1,33

A Tabela 3 evidencia a robustez do randomizado com $\alpha = 0,2$. Na b08, sua média de valor objetivo foi 105,2, correspondente a 1,15% de desvio, enquanto o guloso permaneceu em 108 em todas as execuções. O aumento de α não trouxe benefício: com $\alpha = 0,3$ e $0,4$, as médias foram 107,9 e 109,3, respectivamente. Isso indica que, neste conjunto reduzido de instâncias, ampliar excessivamente a LRC introduziu escolhas de qualidade inferior. O reativo obteve média 107,4 na b08; embora melhor que o guloso, não superou o valor fixo $\alpha = 0,2$. Uma explicação é que a atualização por $1/\bar{f}_i$ produz probabilidades muito parecidas quando as médias dos valores objetivos são próximas, limitando a intensificação no melhor α .

Tabela 4: Tempo médio das dez execuções, em segundos.

Instância	Guloso	Randomizado			Reativo
		$\alpha = 0,2$	$\alpha = 0,3$	$\alpha = 0,4$	
b01	0,5109	0,0822	0,0848	0,0857	0,8168
b03	1,4132	0,1350	0,1369	0,1408	1,4081
b08	4,8049	0,2772	0,2757	0,2799	2,6859
Média	2,2430	0,1648	0,1658	0,1688	1,6369

Os tempos da Tabela 4 crescem com o tamanho do grafo e com a quantidade de terminais. O reativo é mais caro que cada configuração randomizada, o que é esperado porque executa 300 construções, contra 30. Entretanto, os tempos do guloso ficaram

inesperadamente acima dos randomizados em todas as instâncias. Esse resultado não decorre do número de construções e sugere diferenças de custo entre as rotinas internas usadas pelas duas versões; portanto, a comparação de tempo deve ser interpretada como uma medida das implementações atuais, e não apenas das estratégias abstratas. Ainda assim, todos os tempos permaneceram abaixo de cinco segundos, e o randomizado com $\alpha = 0,2$ apresentou o melhor equilíbrio entre qualidade média e tempo neste experimento.

Os resultados devem ser analisados com cautela, pois apenas três instâncias foram avaliadas. Eles demonstram o potencial da randomização, especialmente na **b08**, mas não bastam para afirmar superioridade geral de uma configuração sobre todas as classes de PCSTP.

5 Conclusões

Este trabalho apresentou uma modelagem em grafos e três heurísticas construtivas para o Problema da Árvore de Steiner com Coleta de Prêmios. A implementação reuniu leitura de instâncias STP, representação por listas de adjacência, caminhos mínimos, avaliação da função objetivo, geração pseudoaleatória reproduzível e coleta automatizada de resultados.

Nas instâncias **b01** e **b03**, as estratégias produziram soluções iguais ou muito próximas das melhores conhecidas. Na **b08**, a randomização foi efetivamente útil: $\alpha = 0,2$ reduziu o melhor desvio de 3,85% para 0,96% e o desvio da média para 1,15%. O método reativo apresentou melhora em relação ao guloso nessa instância, mas não superou a melhor parametrização fixa. Assim, dentro do conjunto testado, o randomizado com $\alpha = 0,2$ forneceu a melhor combinação de qualidade, estabilidade e tempo.

As principais dificuldades do projeto foram adaptar o formato STP ao modelo com prêmios, preservar a conectividade durante a construção, evitar duplicação de arestas e organizar um experimento reproduzível com diferentes sementes. Como trabalhos futuros, recomenda-se testar um conjunto maior e mais diversificado de instâncias, incluir uma busca local para remover ramos caros ou substituir caminhos, empregar uma regra reativa que intensifique mais claramente os melhores valores de α e perfilar as rotinas de caminho mínimo para explicar e reduzir o tempo do algoritmo guloso.

Referências

- [1] Michel X. Goemans and David P. Williamson. A general approximation technique for constrained forest problems. *SIAM Journal on Computing*, 24(2):296–317, 1995.
- [2] Thorsten Koch and Alexander Martin. SteinLib: An updated library on Steiner tree problems in graphs. *Steiner Trees in Industry*, 11:285–325, 2001.